Fusion à la Carte:
Modular Compiler Correctness for Effects, a Decade On

Laurence E. Day*

Draft of 24 July 2026

Abstract

The Modular Compilation of Effects (Nottingham, 2017) developed compilers, and virtual machines, that are modular in the features of their source language: syntax, semantics, compilation and execution are all given feature-by-feature and assembled by coproduct. Two problems were left standing. First, the original goal of the programme — per-feature correctness proofs that *compose* into a proof for the whole language — was abandoned as out of reach. Second, effects that fail to commute (exceptions and mutable state being the canonical pair) force a single source program to be compiled differently depending on the order in which monad transformers are applied, an order that is global to the whole language and therefore anti-modular. This paper revisits the programme on foundations that did not exist in 2015: free-monad effect trees and handlers. We show that, once evaluator and machine denote into a *shared* residual effect signature, (i) compiler correctness can be stated as a single equation between effect trees, prior to any choice of handler; (ii) that equation decomposes into per-feature proof obligations which compose by construction, realising the abandoned goal for a language with arithmetic, exceptions, mutable state, output and the untyped λ-calculus; and (iii) the noncommutativity problem does not so much get solved as *relocated*: the local/global state distinction becomes a choice between two elaborations of `catch`, each locally provable, after which handler order is irrelevant to correctness. A mechanically tested Haskell artifact accompanies the paper, the first-order metatheory is mechanised in Agda (checked under `--safe`, without postulates or holes), and a survey keyed to the thesis's own list of open problems traces how the intervening decade supplied each missing piece.

1 Introduction

Between 2010 and 2015 the author pursued a programme, recorded in the thesis *The Modular Compilation of Effects* [9] and the papers preceding it [11, 12, 10], whose aim was to make the intermediate code generation phase of a compiler *modular in the features of the source language*. Syntax was assembled from signature functors by coproduct in the datatypes à la carte style [34]; semantics was assembled from monad transformers in the style of Liang, Hudak and Jones [20]; compilation was given by per-feature algebras of type $f\,(\mathit{Code} \to \mathit{Code}) \to (\mathit{Code} \to \mathit{Code})$ folded over the source; and execution by per-feature algebras of a virtual machine. The thesis delivered a framework in which arithmetic, exceptions, references, the untyped λ-calculus and cyclic control flow could each be defined — syntax, semantics, compilation, execution — in isolation and combined

*Working draft, July 2026. Prepared with machine assistance. The first-order results of §4 (Theorems 4.2–4.4 and their corollaries) are mechanised in the companion Agda development `FusionALaCarte.agda`, checked under `--safe` with no postulates; Theorem 4.6 remains a proof sketch. Comments welcome.

without modification, with structured graphs [22] supplying a target representation for loops.

The thesis is candid about what it did not deliver. Its closing retrospection records that the original goal was

> "that each constituent feature could be proved correct in the style of Wright, and the combination of these proofs would itself constitute a proof for the compound language" [9, §8.1],

a goal in the lineage of *Compiling Exceptions Correctly* [15], and that this was set aside when it was realised "that the truly interesting material lay in the interactions between features themselves." The interaction chiefly responsible is analysed in [9, §5.3]: exceptions and mutable state do not commute, so the expression

```
set 0 (catch (add (set 1 get) throw) get)
```

evaluates to 1 under the "global state" transformer ordering $\mathit{ErrorT}\,(\mathit{StateT}\,\mathbb{Z}\,\mathit{Id})$ and to 0 under the "local" ordering $\mathit{StateT}\,\mathbb{Z}\,(\mathit{ErrorT}\,\mathit{Id})$, and must accordingly be compiled to two *different* instruction sequences — the local one inserting `SAVE`/`RESTORE` instructions around the protected block. Since a compilation algebra is parameterised only by source and target signatures, the thesis was driven to inspect the monad transformer stack itself ("monadic parameterisation", [9, §5.3.2]), techniques it describes as "an alleviation, not a cure". The final chapter's list of future directions ([9, §8.2]) asks, among other things, for reasoning techniques that are "modular along both the source and target languages as well as its computational effects", for a principled account of what it costs to integrate a new feature "based upon the effectful methods it provides", and — via a 2013 suggestion of Swierstra recorded there — whether the ordering of signature functors in a term's type might determine its monadic context automatically.

This paper is the follow-up that the intervening decade makes possible. Algebraic effects and handlers [24, 25] moved from semantic proposal to mainstream implementation technology; scoped and higher-order operations acquired modular theories [37, 27, 40, 5]; compiler correctness acquired compositional, effect-structured foundations in interaction trees [39] and their descendants [8, 42]; and the calculation of compilers from semantics was extended to effectful and concurrent languages [1, 2, 23, 3, 4]. Against that background we make four contributions.

1. **A survey keyed to the thesis's own open problems** (§2). Each item of [9, §8.2] is matched to the line of work that has since addressed it, so that the delta between 2015 and 2026 is stated in the thesis's terms rather than the field's.

2. **Relocation of the noncommutativity problem** (§3). Re-founding the modular semantics on free-monad effect trees, we show that the global/local distinction of [9, §5.3] is not a property of a downstream handler (or transformer) ordering but of the *feature itself*: it is the choice between two elaborations of `catch`, one of which saves and restores the store within its own semantic clause. Once that choice is made per-feature, evaluator and compiler are *handler-order invariant*, answering the "automatic context inference" question by dissolving it.

3. **Compiler correctness as fusion, à la carte** (§4). We state correctness as a single equation between effect trees — prior to any handler — and prove: a componentwise fusion theorem reducing whole-language correctness to independent per-feature obligations (Theorem 4.2); a handler-parametricity theorem showing the equation is preserved by every handler pipeline over the residual signature (Theorem 4.3); and the discharge of

the per-feature obligations for our language, including a complete worked proof for the transactional `catch`, the very case that defeated modularity in 2015 (Theorem 4.4). The untyped λ-calculus is treated by step-guarded iteration, with correctness stated up to fuel (Theorem 4.6, proof sketch).

4. **A tested and partially mechanised artifact** (§5). A self-contained Haskell implementation (548 lines, GHC 9.4, no dependencies) realises the whole pipeline and checks the correctness equation on the thesis's demonstration program, fifteen adversarial edge cases, and 3,600 randomly generated well-scoped terms, each under two handler orders, two initial stores and two fuel budgets, with zero failures. A companion Agda development (767 lines, Agda 2.6.3, `--safe`, no postulates) mechanises the first-order metatheory in full: Theorems 4.2–4.4, the relocation proposition, and the demonstration values under both handler orders, the last by `refl`.

Remark 1.1. Theorems 4.2 and 4.3 are elementary once stated: the first is initial-algebra induction distributed over a coproduct, the second is congruence of handling. We regard the *formulation* — in particular, the choice to state correctness between effect trees over a signature shared by evaluator and machine, so that handler order is quantified out — as the contribution, not the proofs. What the formulation buys is exactly what [9, §8.1] wanted: proof obligations that mention one feature at a time and compose without modification.

2 The landscape since 2015, keyed to §8.2 of the thesis

The thesis closes with seven directions for future work. A decade later, every one of them has a literature. We take them in the thesis's order; Table 1 summarises.

Additional computational features; the algebraic theory of integration cost. The thesis asks for continuations, parallelism and I/O, and "a principled understanding of the complexity involved in integrating a new feature based upon the effectful methods it provides". The second request has since acquired precise content. Plotkin and Pretnar's handlers [24, 25] classify a feature's operations as *algebraic* — those commuting with sequencing, such as `get`, `put`, `throw`, `out` — and everything else. Wu, Schrijvers and Hinze identified the everything-else that matters in practice as *scoped* operations (`catch`, `once`, local binding) [37], for which Piróg et al. gave initial-algebra semantics [27] and Yang et al. a structured handling theory [40]; Yang and Wu subsequently recast the whole hierarchy as monoids with operations [41]. Bach Poulsen and van der Rest's *hefty algebras* [5, 35] then showed that higher-order operations need not be handled directly at all: they can be *elaborated*, modularly and feature-by-feature, into first-order algebraic operations. Latent effects [7] extend the same ambition to λ-abstraction and staging. The integration cost of a feature is, on this account, the height it occupies in the hierarchy algebraic $\subset$ scoped $\subset$ higher-order — a direct, if partial, answer to the thesis's question. Continuations themselves, meanwhile, ceased to be a feature to integrate and became the implementation substrate: handlers subsume delimited control [18], and §3 exploits precisely this.

Automatic context inference. The thesis records Swierstra's suggestion that the coproduct ordering $(\mathit{Arith} : + : \mathit{Except} : + : \mathit{State})$ in a term's type might determine the transformer stack $\mathit{ErrorT} \circ \mathit{StateT}$ used to evaluate it. Row-typed effect systems answered the inference half: in Koka [19], Frank [21], Effekt [6] and their relatives, a term's type carries the (unordered) row of effects it may perform, inferred automatically, and the *handler placement in the program*

text — not a global stack — fixes each operation's semantics. The correspondence between transformer stacks and handler orderings was made precise by Schrijvers et al. [32]. What no type-level ordering can do is choose between local and global state for you, because (as §3 shows) that is a semantic decision belonging to a feature, not a context; the honest resolution of the thesis's question is that half of it is now routine and the other half was asked of the wrong component.

Testing and reasoning. The thesis's hardest request — correctness proofs "modular along both the source and target languages as well as its computational effects", beyond the type-soundness modularity of Delaware et al. [14, 13] — is the one the field has invested in most heavily. Interaction trees [39] are precisely free-monad effect trees equipped with a coinductive silent step and a weak bisimulation, engineered so that compiler correctness becomes an equation between trees, stable under interpretation; they underpin the compositional LLVM semantics of Vellvm [42] and were extended with nondeterministic branching in choice trees [8]. Rouvoet et al. showed that even the grubby target-side details — labels, jumps, bytecode well-formedness — admit intrinsically-typed, separation-logic-structured treatment [29], and DimSum [33] gave a decentralised account of linking across languages. None of these, however, organises its proofs *per source-language feature*: the unit of modularity is the pass, the module or the language, not the signature functor. Section 4 is our attempt to occupy exactly that gap, using the interaction-tree insight (state correctness between trees, quantify the interpretation out) at à la carte granularity.

Alternative target languages; dataflow and optimisation. Bahr and Hutton's calculation series took the semantics-to-machine derivation the thesis performed by hand and made it systematic: for pure languages [1], for register machines [2], with dependent types [23], for effectful source languages via computation monads [3], and for concurrency [4]. Their method calculates a *whole* compiler from a whole semantics; ours composes a compiler from feature fragments. The two are complementary, and §7 sketches the obvious marriage. On the target side, the thesis's stack machines and structured graphs [22, 10] have an unexpected successor: the effect-handler constructs themselves are now compilation targets, via capability-passing [30], evidence passing [38], multicore OCaml's runtime fibres [31], and WebAssembly's typed continuations proposal [26]. A machine that natively unwinds to handler frames — as ours does for `catch` — is no longer a pedagogical device but a portrait of a production runtime.

3 Effect trees, and where the noncommutativity went

3.1 Programs over a signature

We write $\mathsf{Prog}\,\sigma\,a$ for the free monad over a signature functor σ: a leaf $\mathsf{Return}\,a$, or an operation node $\mathsf{Op}\,(\sigma\,(\mathsf{Prog}\,\sigma\,a))$. Signatures compose by the same coproduct $(:+:)$ as source syntax, with injection provided by the usual subsumption class. Four algebraic signatures suffice for this paper, shared — and this is the load-bearing decision — by evaluator and machine alike:

```
data StateF k = Get (Int → k) | Put Int k      -- a single mutable cell
data OutF   k = Out Int k                       -- observable output
data StepF  k = Step k                          -- one unit of fuel
data ExcF   k = Throw'                          -- abort (no continuation)
```

Open problem, [9, §8.2]	Where the decade took it
Additional features; cost of integrating a feature	Algebraic/scoped/higher-order hierarchy [37, 27, 40, 41]; modular elaboration via hefty algebras [5, 35]; latent effects [7]
Automatic context inference	Row-typed effects and handler placement [19, 21, 6]; transformer/handler correspondence [32]; residual semantic choice relocated per-feature (§3)
Testing and reasoning	Interaction and choice trees [39, 8]; Vellvm [42]; intrinsically-typed compilation [29]; DimSum [33]; this paper's §4 at feature granularity
Alternative targets	Calculated register machines [2]; effect handlers as targets: capability/evidence passing [30, 38], OCaml 5 [31], WasmFX [26]
Attribute grammars; modular syntax; indexed families	Subsumed in practice by the effect-signature discipline; higher-order functor syntax standard since [16]; elaborations play the source-to-target type-family role
Dataflow and optimisation	Fusion of handlers [36, 17]; graph-based targets in the calculational line [4]

Table 1: The thesis's future-work list, mapped to 2015–2026 developments.

```
type Sig  = StateF :+: OutF :+: StepF :+: End    -- machine-side residue
type ESig = ExcF :+: Sig                          -- evaluator-side residue
```

Handlers peel one signature component and interpret it: $\mathsf{handleSt} : \mathbb{Z} \to \mathsf{Prog}\,(\mathit{StateF} :+: r)\,a \to \mathsf{Prog}\,r\,(a \times \mathbb{Z})$ and similarly handleOut, handleFuel, handleExc, with $\mathsf{run} : \mathsf{Prog}\,\mathit{End}\,a \to a$ closing the pipeline. Because coproducts of signatures are symmetric monoidal, adjacent components can be exchanged by an evident natural isomorphism (swapSO in the artifact); "handler order" is therefore a property of a pipeline, not of a program.

3.2 The demo, redone

The source language is assembled exactly as in [9, Ch. 4–5]: signature functors *Arith*, *Excep*, *Mem*, *Print*, *Lam*, a fixpoint of their coproduct, and one evaluation algebra per feature, each folding into $\mathit{Env} \to \mathsf{Prog}\,\mathsf{ESig}\,\mathit{Value}$. The single but decisive departure from the thesis is that the exception feature now owns *two* scoped constructors, and gives each its semantics locally, using the interception combinator $\mathsf{catchP}\,t\,h$ that replaces the leftmost Throw' leaves of t by h and passes all residual operations through:

```
instance Eval Excep where
  evalAlg Throw        env = throwP
  evalAlg (CatchG x h) env = catchP (x env) (h env)
  evalAlg (CatchL x h) env = do { s ← getP
                                ; catchP (x env) (putP s ≫ h env) }
```

CatchG is the global-state `catch`: store updates made in the protected block survive an abort, because the Put nodes were emitted into the tree before the abort was intercepted. *CatchL* is the transactional `catch` of [9, §5.3]: it reads the store on entry and restores it on the failure path — *inside its own clause.* In hefty-algebra terms [5], these are two elaborations of a higher-order operation into the first-order signature ESig; in the thesis's terms, the `SAVE/RESTORE` pair has moved from the compiled code (where it needed knowledge of the transformer stack to place) into the feature's definition (where it needs none).

Proposition 3.1 (Relocation of noncommutativity)**.** *Let e range over the source language above and let $\mathcal{H}_1, \mathcal{H}_2$ be any two handler pipelines over* Sig *that differ by a permutation of signature components, applied after* handleExc*. Then $\mathcal{H}_1(\mathsf{handleExc}\,(\mathsf{eval}\,e\,\rho))$ and $\mathcal{H}_2(\mathsf{handleExc}\,(\mathsf{eval}\,e\,\rho))$*

agree up to the result-type isomorphism induced by the permutation. In particular, for the thesis's demonstration program `set 0 (catch (add (set 1 get) throw) get)`*, the value* 1 *arises iff* `catch` *is read as CatchG and* 0 *iff as CatchL, independently of pipeline order.*

Proof. The first claim is Theorem 4.3 applied to the trivial equation $t = t$ together with naturality of the exchange isomorphisms. The second is a finite calculation on the two trees; the artifact checks it under both orders (§5). □

The point is small but, we think, clarifying. The thesis experienced noncommutativity as a *global* obstruction: one program, one compilation algebra, two intended semantics, disambiguated only by a whole-stack type. On effect trees the ambiguity cannot even be expressed: a program is a tree of operations, its meaning is fixed once each scoped operation has said what it does, and downstream handler order can only re-nest results. What remains genuinely semantic — do you want transactions? — is answered where it should be, in the feature. Swierstra's inference question thus splits: effect *membership* is inferred by row typing [19, 32]; effect *interaction* is not inferred at all, because it is data.

4 Compilation as fusion

4.1 The pipeline

Target code is a fixpoint of instruction functors, one per source feature plus structural glue, and the compiler is the thesis's [9, §4.3] fold of per-feature algebras with code-continuation carrier — unchanged up to the two `catch` schemes now being ordinary, stack-inspecting instruction sequences rather than transformer-directed ones:

```
class Functor f ⇒ Comp f where
  compAlg :: f (Code → Code) → Code → Code

instance Comp Excep where
  compAlg Throw        = const throwc
  compAlg (CatchG x h) = λc → markc (h c) (x (unmarkc c))
  compAlg (CatchL x h) = λc →
      loadc (markc (storec (h c)) (x (unmarkc (slidec c))))
```

CatchL compiles to: push the current store (`LOAD`); install a handler frame whose code begins by popping that value back into the store (`MARK`(`STORE`; h)); run the block; on success remove the frame (`UNMARK`) and the now-unneeded snapshot (`SLIDE`). The machine is a family of per-feature execution algebras over configurations $Conf = Stack \times MEnv$, written in open recursion and producing trees in Prog Sig *Conf*: `LOAD`/`STORE` emit Get/Put, `OUT` emits Out, `CALL` emits Step, and `THROW` performs the classic Hutton–Wright unwinding [15] to the nearest handler frame — restoring the frame's saved environment — or, if none, to a distinguished failed configuration. Exceptions are thus *compiled away* (the machine's residue is Sig, not ESig), state and output are *kept abstract* (the machine no more chooses a store implementation than the evaluator does), and steps are *synchronised* (one Step per β-step on both sides).

4.2 The correctness equation

To compare a Prog ESig *Value* tree with a Prog Sig *Conf* tree we read the former through the machine's eyes. Fix a value translation $\widehat{(\cdot)}$ (numbers to numbers; closures per §4.4)

and an environment relation $\rho \approx env$. Define $\mathsf{interp} : \mathsf{Prog}\,\mathsf{ESig}\,\mathit{Value} \to \mathit{Code} \to \mathit{Conf} \to \mathsf{Prog}\,\mathsf{Sig}\,\mathit{Conf}$ by

$$\begin{aligned} \mathsf{interp}\,(\mathsf{Return}\,v)\;c\;(k, env) &= \mathsf{exec}\;c\;(\widehat{v} :: k,\; env) \\ \mathsf{interp}\,(\mathsf{Op}\,(\mathsf{Inl}\,\mathsf{Throw}'))\;c\;(k, env) &= \mathsf{unwind}\;k \\ \mathsf{interp}\,(\mathsf{Op}\,(\mathsf{Inr}\,op))\;c\;\kappa &= \mathsf{Op}\,(\mathsf{fmap}\;(\lambda t.\;\mathsf{interp}\;t\;c\;\kappa)\;op) \end{aligned}$$

interp is itself a handler: it interprets the exception component into stack unwinding and passes the shared residue through untouched. Correctness is then one equation.

Definition 4.1 (Feature-local soundness)**.** *Let $A_c = \mathit{Code} \to \mathit{Code}$ and $A_e = \mathit{Env} \to \mathsf{Prog}\,\mathsf{ESig}\,\mathit{Value}$ be the compiler and evaluator carriers, and define $P \subseteq A_c \times A_e$ by: $P(\kappa, \delta)$ iff for all code c, configurations (k, env) and environments $\rho \approx env$,*

$$\mathsf{exec}\;(\kappa\;c)\;(k, env) \;=\; \mathsf{interp}\;(\delta\;\rho)\;c\;(k, env). \tag{S}$$

A feature F (a signature functor with algebras $compAlg_F$, $evalAlg_F$) is locally sound *if the paired algebra preserves P: whenever an F-structure is built from pairs satisfying P, the pair $(compAlg_F\;x,\; evalAlg_F\;y)$ of its images satisfies P.*

Two remarks. First, (S) quantifies over *all* code continuations and configurations, so it is compositional by design — this is the compiler-correctness analogue of stating an interaction-tree simulation for all contexts [39]. Second, and crucially, the only vocabulary a local soundness proof may use is the feature's own constructors plus a fixed *machine interface*: exec on continuation code, unwind, $\widehat{(\cdot)}$, $\approx$, and the invariant — baked into interp's Return clause — that compiled subterms restore the machine environment. That interface is the formal residue of what [9, §8.1] called "the style of Wright"; making it explicit is what lets the proofs compose.

Theorem 4.2 (Fusion à la carte)**.** *Let $F_1, \ldots, F_n$ be locally sound features. Then (S) holds of the whole language: for every $e \in \mu(F_1 :+: \cdots :+: F_n)$, all c, (k, env) and $\rho \approx env$,*

$$\mathsf{exec}\;(\mathsf{fold}\,compAlg\;e\;c)\;(k, env) = \mathsf{interp}\;(\mathsf{fold}\,evalAlg\;e\;\rho)\;c\;(k, env).$$

Proof. Fold the paired algebra $\langle compAlg, evalAlg\rangle$ over the initial algebra $\mu(\Sigma F_i)$ with carrier $A_c \times A_e$; the two projections of this fold are comp and eval by uniqueness of folds. P is a predicate on the carrier; by initial-algebra induction (the fold factors through any P-closed subalgebra), P holds of the fold's image provided the paired algebra preserves P. The paired algebra of a coproduct is the copair of the components' paired algebras, so preservation of P by the copair is precisely the conjunction of preservation by each component — which is local soundness of each F_i. Note the theorem's proof mentions no feature; all feature-specific content lives in the local obligations. □

Theorem 4.3 (Handler parametricity)**.** *If (S) holds, then for every handler pipeline $\mathcal{H}$ over* Sig *— in particular for every interleaving of* handleSt s_0, handleOut *and* handleFuel n *obtained via the exchange isomorphisms, for every initial store s_0 and fuel budget n — we have $\mathcal{H}(\mathsf{exec}\,(\mathsf{comp}\,e)\,\kappa_0) = \mathcal{H}(\mathsf{interp}\,(\mathsf{eval}\,e\,\rho_0)\,\mathsf{HALT}\,\kappa_0)$.*

Proof. Handlers and exchange isomorphisms are functions; equality is a congruence. The content is in what the statement rules out: no choice made *after* the trees are formed — ordering, store, budget, indeed any Sig-algebra whatsoever, by the universal property of the free monad — can separate compiled code from source semantics. The thesis's transformer-order ambiguity has nowhere left to live. □

4.3 Discharging the local obligations

Theorem 4.4 (Local soundness of the first-order features)**.** *Arith, Mem, Print and Excep (both* `catch` *schemes) are locally sound.*

Proof. *Arith*, *Mem* and *Print* are short calculations of an identical shape; we show the interesting case, *CatchL*, in full, and note that *CatchG* is the same proof minus the store bookkeeping. Assume $P(\hat{x}, x)$ and $P(\hat{h}, h)$; fix c, (k, env), $\rho \approx env$. Abbreviate $F = \mathsf{IHan}\,(\mathtt{STORE}(\hat{h}\,c))\,env$. On the left,

$$\begin{aligned}
&\mathsf{exec}\,(\mathtt{LOAD}(\mathtt{MARK}\,(\mathtt{STORE}(\hat{h}\,c))\,(\hat{x}\,(\mathtt{UNMARK}(\mathtt{SLIDE}\,c)))))\,(k, env)\\
&\quad= \mathsf{Op}\big(\mathsf{Get}\,\lambda s.\,\mathsf{exec}\,(\hat{x}\,(\mathtt{UNMARK}(\mathtt{SLIDE}\,c)))\,(F :: \mathsf{IN}\,s :: k,\ env)\big) && [\mathtt{LOAD}, \mathtt{MARK}]\\
&\quad= \mathsf{Op}\big(\mathsf{Get}\,\lambda s.\,\mathsf{interp}\,(x\,\rho)\,(\mathtt{UNMARK}(\mathtt{SLIDE}\,c))\,(F :: \mathsf{IN}\,s :: k,\ env)\big) && [P(\hat{x}, x)]
\end{aligned}$$

while on the right, unfolding *evalAlg* and interp's pass-through clause,

$$\mathsf{interp}\,(\mathit{evalAlg}(\mathit{CatchL})\,\rho)\,c\,(k, env) = \mathsf{Op}\big(\mathsf{Get}\,\lambda s.\,\mathsf{interp}\,(\mathsf{catchP}\,(x\,\rho)\,(\mathsf{putP}\,s \gg h\,\rho))\,c\,(k, env)\big).$$

It therefore suffices to prove, for all t and all s,

$$\mathsf{interp}\,(\mathsf{catchP}\,t\,(\mathsf{putP}\,s \gg h\,\rho))\,c\,(k, env) \;=\; \mathsf{interp}\,t\,(\mathtt{UNMARK}(\mathtt{SLIDE}\,c))\,(F :: \mathsf{IN}\,s :: k,\ env),$$

by induction on t. *Case* $\mathsf{Return}\,v$*:* the left side is $\mathsf{exec}\,c\,(\widehat{v} :: k, env)$; the right side steps `UNMARK` (dropping F beneath the top) then `SLIDE` (dropping $\mathsf{IN}\,s$), reaching $\mathsf{exec}\,c\,(\widehat{v} :: k, env)$. *Case* $\mathsf{Op}(\mathsf{Inl}\,\mathsf{Throw}')$*:* the left side is $\mathsf{interp}\,(\mathsf{putP}\,s \gg h\,\rho)\,c\,(k, env) = \mathsf{Op}(\mathsf{Put}\,s\,(\mathsf{interp}\,(h\,\rho)\,c\,(k, env))) = \mathsf{Op}(\mathsf{Put}\,s\,(\mathsf{exec}\,(\hat{h}\,c)\,(k, env)))$ using $P(\hat{h}, h)$; the right side unwinds past no frames to F, recovering stack $\mathsf{IN}\,s :: k$ and environment env, and executes $\mathtt{STORE}(\hat{h}\,c)$, which pops $\mathsf{IN}\,s$, emits $\mathsf{Put}\,s$, and continues with $\mathsf{exec}\,(\hat{h}\,c)\,(k, env)$ — the same tree. *Case* $\mathsf{Op}(\mathsf{Inr}\,op)$*:* both sides wrap $\mathsf{Op}(\mathsf{Inr}\,\cdot)$ around the same functorial image and the inductive hypothesis applies pointwise; note this is exactly the clause through which the protected block's own Puts pass untouched on both sides, which is why *CatchG* is global and *CatchL*'s snapshot is the *only* transactional ingredient. The unwinding on the right discards any items the block pushed above F — including return frames, which is why exceptions correctly cross function calls — and restores F's saved environment, matching the left side's use of the entry-time ρ. □

Corollary 4.5. *The thesis's demonstration program compiles correctly under both readings of* `catch`*, yielding* 1 *(CatchG) and* 0 *(CatchL) under* every *handler pipeline over* Sig*.*

Contrast with [9, §5.3.2]: there, the choice of scheme was recovered by pattern-matching on the transformer stack, a mechanism the thesis itself judged "an alleviation, not a cure", and no correctness statement relating the two compiled forms to the two monads was available. Here each scheme carries its own ten-line proof, the proofs share the block's obligation $P(\hat{x}, x)$ verbatim, and the monad has disappeared from the statement entirely.

4.4 The higher-order fragment

λ-abstraction is the feature the thesis could compile ([9, §5.2], via closures, with call-by-value and call-by-name schemes) but not fit into any correctness story. Two of the 2015-era obstacles have dissolved: higher-order syntax is now routinely handled by elaboration [5, 7], and the semantic gap — evaluation of an application is not structural recursion — is addressed by the interaction-tree device of a silent step [39]. We adopt both: eval emits one Step per application, the machine emits one per `CALL`, and correctness is stated *up to fuel*.

Values now include closures, and the translation $\widehat{(\cdot)}$ can no longer be defined by recursion on values: relating VFun f to IClo *code cenv* requires relating behaviours of bodies, which mentions the relation being defined at a mixed variance. We therefore index the relations by a step budget in the standard way: $v \approx_m i$ for closures holds iff for all $m' < m$ and arguments related at m', the instantiated bodies satisfy (S) truncated at m' steps, where truncation is handleFuel after reordering StepF to the front.

Theorem 4.6 (Local soundness of *Lam*, up to fuel)**.** *For every m, the Lam algebras preserve the m-indexed version of P; consequently, for the full language,* $\mathsf{handleFuel}_m$ *applied (after exchange) to the two sides of* (S) *agree for every m, and hence so do all further handler pipelines.*

Proof sketch. Var and Abs are immediate ($\approx$ on environments is pointwise; an abstraction's closure and its compiled `CLOSE` capture related environments and are related at every index by the induction being set up). For App, both sides evaluate function then argument (obligations $P(\hat{f}, f)$, $P(\hat{a}, a)$ at index m), then spend one Step synchronously — so the truncations agree on the prefix, and the budgets entering the bodies agree at $m' < m$ — after which the closure relation at m' yields exactly the required equation for the body, with the machine's IRet frame discharging the interface invariant that the caller's environment is restored, and the dynamic-error cases (β-redex whose head is a number) reducing to the exception interface on both sides. The argument is the standard step-indexed logical relation transposed to effect trees; a full treatment belongs in a proof assistant, for which interaction trees' eutt [39] or guarded variants are the natural home, and is the one result the companion Agda development (§5) does not yet cover. The artifact tests the truncated statement directly, including at deliberately tiny budgets where any desynchronisation of Steps between the two sides would be observable, and on divergent terms such as $(\lambda x.\, x\, x)(\lambda x.\, x\, x)$. □

5 The artifact

The companion file `FusionALaCarte.hs` (548 lines, GHC 9.4, `base` only) implements everything above: the à la carte infrastructure, Prog and its handlers including the exchange isomorphism, the five-feature source language, the two-scheme compiler, and the machine with handler-frame unwinding, closures, return frames and synchronised steps. Its test driver checks the equation of Theorem 4.3 at the top-level continuation (inner instances of (S) are exercised through enclosing contexts, since generated terms nest features freely):

- the thesis demonstration program, both `catch` schemes, asserting the values $1/0$ under both handler orders;
- fifteen edge cases: nested mixed catches, exceptions thrown across function calls, state threaded through calls, closures as results, dynamic type errors (applying a number, adding a closure) interacting with `catch`, output emitted inside aborted transactional blocks (correctly *not* rolled back: output has no local scheme), variable shadowing, and Ω under fuel;
- 3,600 randomly generated well-scoped terms in three size bands, each compared under two handler orders (St-then-Out and the exchange), two initial stores, and fuel budgets $\{400, 4\}$ — the tiny budget makes any step-count mismatch between evaluator and machine observable.

All 3,621 checks pass. Testing establishes, of course, only the instances it runs; its role here is the one [9, §8.2] assigned it — modular generation of programs and configurations as

a cheap conscience for the proofs — and the failure of any single seed would have falsified Theorems 4.4 or 4.6 as implemented.

The Agda development. The companion file `FusionALaCarte.agda` (767 lines, Agda 2.6.3, standard library 1.7.3) mechanises the first-order fragment end to end, and typechecks under `--safe` with no postulates or holes. Signatures are containers, so all functors are strictly positive by construction, and tree equality is an inductive pointwise bisimilarity rather than propositional equality under function extensionality; the eight demonstration equations of Proposition 3.1 then hold literally by `refl`. One presentational delta from the Haskell artifact is forced by totality: the mechanised machine carries the *meanings* of handler frames on a separate failure-continuation stack, so that `exec` is structurally recursive on code, while the Haskell machine stores frame *code* in the value stack and unwinds through it. The latter is the defunctionalisation of the former in Reynolds's sense [28], and the two agree on all compiled code; the fold-friendly presentation is precisely the "meaning-carrying frames" anticipated in §4. The statement mechanised is Definition 4.1 verbatim — `fusion : (e : Expr) → P (fold compAlg e) (fold evalAlg e)` — with $\oplus$`-sound` and μ`-sound` as the two feature-free halves of Theorem 4.2, one `LocallySound` record per feature for Theorem 4.4, and per-handler $\approx$-congruence lemmas assembling Theorem 4.3 into the corollaries `agreeA`/`agreeB`: compiled code and source semantics agree under both handler orders, for every initial store and fuel budget.

6 Related work

Beyond the survey of §2: the idea that a compiler is a fold and correctness a fusion property is as old as Meijer's thesis-era slogans and is the engine of the Bahr–Hutton calculation line [1, 2, 3]; our Theorem 4.2 differs in distributing the fusion obligation over a coproduct of features against a fixed machine interface, rather than calculating one machine from one semantics. Modular Monadic Meta-theory [13] composes *type-soundness* proofs per feature over monad-transformer semantics and identified effect interaction as the obstacle; replacing the transformer stack by a shared free signature is what removes that obstacle here, at the price of fixing the operations (not their interpretations) up front. Handler fusion [36, 17] fuses *interpreters* for performance; we fuse an interpreter with a compiler for correctness — the algebra is the same, the theorem is different. Interaction trees [39, 42, 8] supply the state-it-between-trees discipline and the silent step we borrow; what they do not supply, and we add, is the per-signature-functor decomposition of the compiler-correctness obligation. Finally, the machine that unwinds to marked frames descends directly from [15] via [9, Ch. 7], and its contemporary significance is that production runtimes now look like it [31, 26].

7 Conclusion, and the next decade's list

The thesis asked whether per-feature compiler correctness proofs could compose, and answered "not yet". On effect trees the answer is yes, for a language whose features span pure computation, aborts, scoped transactions, mutable state, observable output and untyped λ-abstraction — with the noncommutativity that blocked the 2015 attempt reduced to a pair of independently provable elaborations. In the spirit of [9, §8.2], we close with our own list.

Mechanisation. Theorems 4.2–4.4 turned out to be exactly the finite equational arguments they appear to be: the companion Agda development consumed them whole (§5), and

the transliteration from the Haskell artifact was a day's work rather than a project. What remains is the step-indexed Theorem 4.6, which wants interaction trees' eutt [39] or a guarded logic, and the formal correspondence between the fold-friendly machine and its defunctionalised, frame-in-stack image.

Cyclic control. The thesis's structured-graph targets [22, 10] are conspicuously absent above. The interp-style statement should extend: a graph target makes exec a genuine iteration, and the fuel discipline of §4.4 already speaks that language; the calculational treatment of graph-shaped machines in the Bahr–Hutton line [4] is the obvious partner.

Register machines and real targets. Local soundness against the stack interface should be re-provable against the register interface of [2]; more ambitiously, against WasmFX [26], where handler frames are not a model of the runtime but the runtime.

The cost question, properly. §2 answered the thesis's "complexity of integrating a feature" question by hierarchy position (algebraic/scoped/higher-order). The fusion framing suggests a sharper metric: the size of a feature's local soundness obligation — which machine-interface operations it touches. *Arith* touches the stack; *CatchL* touches frames and the store signature; *Lam* touches everything and costs a step index. Making that metric precise, and relating it to the elaboration hierarchy, seems both tractable and worth doing.

Acknowledgements. The debts of the 2017 thesis stand; particular thanks, across the decade, to Graham Hutton and Patrick Bahr, whose calculation series kept the question alive.

References

[1] P. Bahr and G. Hutton. Calculating correct compilers. *Journal of Functional Programming*, 25, 2015.

[2] P. Bahr and G. Hutton. Calculating correct compilers II: Return of the register machines. *Journal of Functional Programming*, 30, 2020.

[3] P. Bahr and G. Hutton. Monadic compiler calculation (functional pearl). *Proc. ACM Program. Lang.*, 6(ICFP), 2022.

[4] P. Bahr and G. Hutton. Calculating compilers for concurrency. *Proc. ACM Program. Lang.*, 7(ICFP), 2023.

[5] C. Bach Poulsen and C. van der Rest. Hefty algebras: Modular elaboration of higher-order algebraic effects. *Proc. ACM Program. Lang.*, 7(POPL), 2023.

[6] J. I. Brachthäuser, P. Schuster and K. Ostermann. Effekt: Capability-passing style for type- and effect-safe, extensible effect handlers. *Journal of Functional Programming*, 30, 2020.

[7] B. van den Berg, T. Schrijvers, C. Bach Poulsen and N. Wu. Latent effects for reusable language components. In *APLAS*, 2021.

[8] N. Chappe, P. He, L. Henrio, E. Ioannidis, Y. Zakowski and S. Zdancewic. Choice trees: Representing nondeterministic, recursive, and impure programs in Coq. *Proc. ACM Program. Lang.*, 7(POPL), 2023.

[9] L. E. Day. *The Modular Compilation of Effects*. PhD thesis, University of Nottingham, 2017.

[10] L. E. Day and P. Bahr. Pick'n'Fix: Capturing control flow in modular compilers. In *Trends in Functional Programming (TFP)*, 2014.

[11] L. E. Day and G. Hutton. Towards modular compilers for effects. In *Trends in Functional Programming (TFP)*, 2011.

[12] L. E. Day and G. Hutton. Compilation à la carte. In *Implementation and Application of Functional Languages (IFL)*, 2013.

[13] B. Delaware, S. Keuchel, T. Schrijvers and B. C. d. S. Oliveira. Modular monadic meta-theory. In *ICFP*, 2013.

[14] B. Delaware, B. C. d. S. Oliveira and T. Schrijvers. Meta-theory à la carte. In *POPL*, 2013.

[15] G. Hutton and J. J. Wright. Compiling exceptions correctly. In *Mathematics of Program Construction (MPC)*, 2004.

[16] P. Johann and N. Ghani. Foundations for structured programming with GADTs. In *POPL*, 2008.

[17] O. Kiselyov and H. Ishii. Freer monads, more extensible effects. In *Haskell Symposium*, 2015.

[18] O. Kammar, S. Lindley and N. Oury. Handlers in action. In *ICFP*, 2013.

[19] D. Leijen. Type directed compilation of row-typed algebraic effects. In *POPL*, 2017.

[20] S. Liang, P. Hudak and M. P. Jones. Monad transformers and modular interpreters. In *POPL*, 1995.

[21] S. Lindley, C. McBride and C. McLaughlin. Do be do be do. In *POPL*, 2017.

[22] B. C. d. S. Oliveira and W. R. Cook. Functional programming with structured graphs. In *ICFP*, 2012.

[23] M. Pickard and G. Hutton. Calculating dependently-typed compilers (functional pearl). *Proc. ACM Program. Lang.*, 5(ICFP), 2021.

[24] G. D. Plotkin and M. Pretnar. Handlers of algebraic effects. In *ESOP*, 2009.

[25] G. D. Plotkin and M. Pretnar. Handling algebraic effects. *Logical Methods in Computer Science*, 9(4), 2013.

[26] L. Phipps-Costin, A. Rossberg, A. Guha, D. Leijen, D. Hillerström, K. C. Sivaramakrishnan, M. Pretnar and S. Lindley. Continuing WebAssembly with effect handlers. *Proc. ACM Program. Lang.*, 7(OOPSLA2), 2023.

[27] M. Piróg, T. Schrijvers, N. Wu and M. Jaskelioff. Syntax and semantics for operations with scopes. In *LICS*, 2018.

[28] J. C. Reynolds. Definitional interpreters for higher-order programming languages. In *Proceedings of the ACM Annual Conference*, 1972. Reprinted in *Higher-Order and Symbolic Computation*, 11(4), 1998.

[29] A. Rouvoet, R. Krebbers and E. Visser. Intrinsically typed compilation with nameless labels. *Proc. ACM Program. Lang.*, 5(POPL), 2021.

[30] P. Schuster, J. I. Brachthäuser and K. Ostermann. Compiling effect handlers in capability-passing style. *Proc. ACM Program. Lang.*, 4(ICFP), 2020.

[31] K. C. Sivaramakrishnan, S. Dolan, L. White, T. Kelly, S. Jaffer and A. Madhavapeddy. Retrofitting effect handlers onto OCaml. In *PLDI*, 2021.

[32] T. Schrijvers, M. Piróg, N. Wu and M. Jaskelioff. Monad transformers and modular algebraic effects: what binds them together. In *Haskell Symposium*, 2019.

[33] M. Sammler, S. Spies, Y. Song, E. D'Osualdo, R. Krebbers, D. Garg and D. Dreyer. DimSum: A decentralized approach to multi-language semantics and verification. *Proc. ACM Program. Lang.*, 7(POPL), 2023.

[34] W. Swierstra. Data types à la carte. *Journal of Functional Programming*, 18(4), 2008.

[35] C. van der Rest and C. Bach Poulsen. Hefty algebras: Modular elaboration of higher-order effects. *Journal of Functional Programming*, 35, 2025.

[36] N. Wu and T. Schrijvers. Fusion for free: Efficient algebraic effect handlers. In *Mathematics of Program Construction (MPC)*, 2015.

[37] N. Wu, T. Schrijvers and R. Hinze. Effect handlers in scope. In *Haskell Symposium*, 2014.

[38] N. Xie and D. Leijen. Generalized evidence passing for effect handlers: Efficient compilation of effect handlers to C. *Proc. ACM Program. Lang.*, 5(ICFP), 2021.

[39] L.-y. Xia, Y. Zakowski, P. He, C.-K. Hur, G. Malecha, B. C. Pierce and S. Zdancewic. Interaction trees: Representing recursive and impure programs in Coq. *Proc. ACM Program. Lang.*, 4(POPL), 2020.

[40] Z. Yang, M. Paviotti, N. Wu, B. van den Berg and T. Schrijvers. Structured handling of scoped effects. In *ESOP*, 2022.

[41] Z. Yang and N. Wu. Modular models of monoids with operations. *Proc. ACM Program. Lang.*, 7(ICFP), 2023.

[42] Y. Zakowski, C. Beck, I. Yoon, I. Zaichuk, V. Zaliva and S. Zdancewic. Modular, compositional, and executable formal semantics for LLVM IR. *Proc. ACM Program. Lang.*, 5(ICFP), 2021.